



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE





Do Código ao Container

Entenda o problema real que o Docker resolve, aprenda os conceitos fundamentais e comece a escrever Dockerfiles para backends e frontends modernos.



O Que Vamos Aprender Hoje

01

O Problema

Por que "na minha máquina funciona" é um risco técnico real

02

Conceitos Fundamentais

Imagem vs. container — qual a diferença e por que importa

03

Anatomia do Dockerfile

As instruções essenciais para descrever seu ambiente

04

Dockerfiles Reais

Backend Node.js/Express e frontend React com multi-stage build

05

Ciclo de Vida Básico

Build, run, ps, logs e stop na prática





Antes de qualquer conceito

**"Alguém do time
tentou rodar o projeto
de outro colega esse
semestre e algo
quebrou. O que foi?"**

Esse problema tem nome. E tem solução.



"Na Minha Máquina Funciona"

Essa frase é uma das mais temidas no desenvolvimento de software. O código roda perfeitamente no seu notebook, mas quebra em produção, ou na máquina de outro desenvolvedor da equipe. Por que isso acontece?

Versões Diferentes

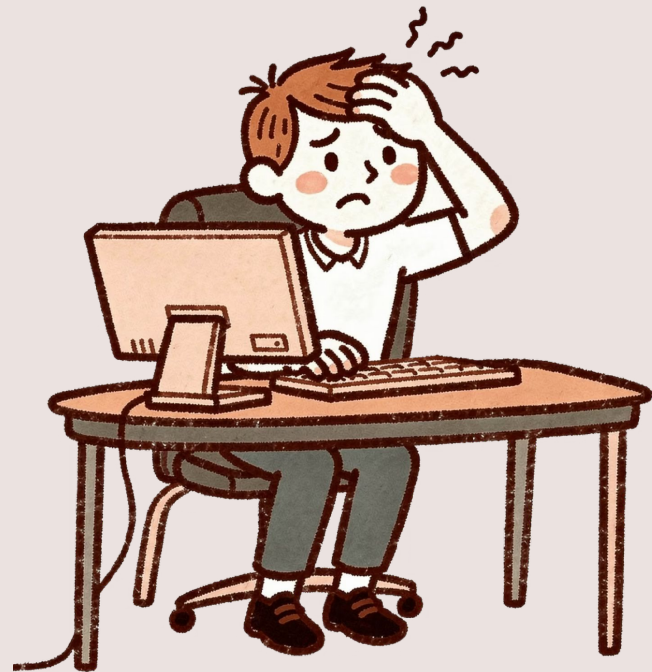
Node 16 no dev, Node 18 em produção. Uma mudança de versão pode quebrar dependências silenciosamente.

Variáveis de Ambiente

Configurações locais que nunca foram documentadas: portas, credenciais, caminhos de sistema.

Sistema Operacional

Windows, macOS e Linux têm comportamentos distintos em caminhos, permissões e binários nativos.



A Raiz Técnica do Problema

Ambiente Implícito

Toda aplicação depende de um conjunto de software ao seu redor: runtime, bibliotecas do SO, variáveis de configuração e serviços externos. Esse ambiente raramente é descrito de forma explícita e versionada, ele simplesmente existe na máquina de quem desenvolveu.

O Resultado Prático

Bugs que aparecem apenas em produção, onboarding lento de novos devs (horas configurando o ambiente), pipelines de CI que se comportam de forma diferente do local, e deploys imprevisíveis.



A solução não é documentar melhor o README, é tornar o ambiente parte do código.



Docker empacota o ambiente junto com o código.

Não apenas as dependências do `package.json`, mas o runtime, o sistema de arquivos e as configurações necessárias para rodar a aplicação em qualquer lugar.



Centro de
Informática
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO



Docker empacota código + ambiente juntos



Uma imagem Docker é imutável, portátil e versionável — como um artefato de software que carrega tudo o que precisa para rodar. A partir de uma única imagem, você pode gerar dezenas de containers idênticos rodando em qualquer ambiente: máquina do dev, CI/CD, staging ou produção.

Imagem

O pacote (a receita): imutável, portátil, versionável. Construída via Dockerfile.

Container

A instância em execução (o bolo assado a partir da receita). Tem processo, memória e estado.



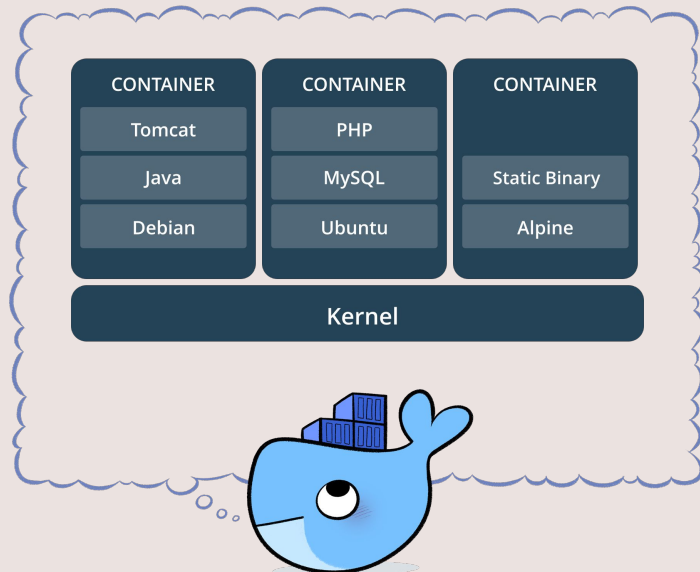
O que são contêineres?

- CONTÊINERES SÃO PROCESSOS
- Contêineres são processos isolados por:
 - Namespaces do kernel
 - Gerenciamento de privilégios de root
 - Restrições de chamadas de sistema
 - Pilhas de rede privadas
 - etc.



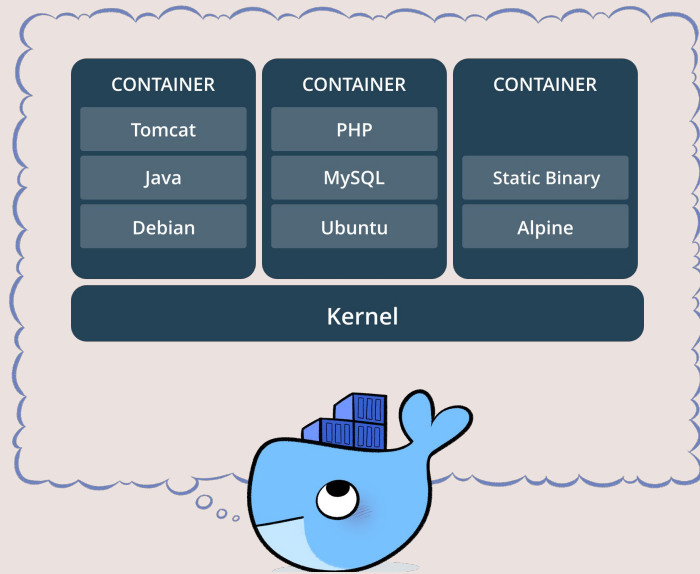
O que são contêineres?

- Os contêineres fornecem espaços privados sobre o sistema operacional.
- Essa técnica também é chamada de virtualização do sistema operacional.
 - O kernel do sistema operacional fornece espaços virtuais isolados.
 - Cada um desses espaços virtuais é chamado de contêiner ou mecanismo virtual (VE).
- Os contêineres permitem que os processos sejam executados em um ambiente isolado sobre o sistema operacional hospedeiro.

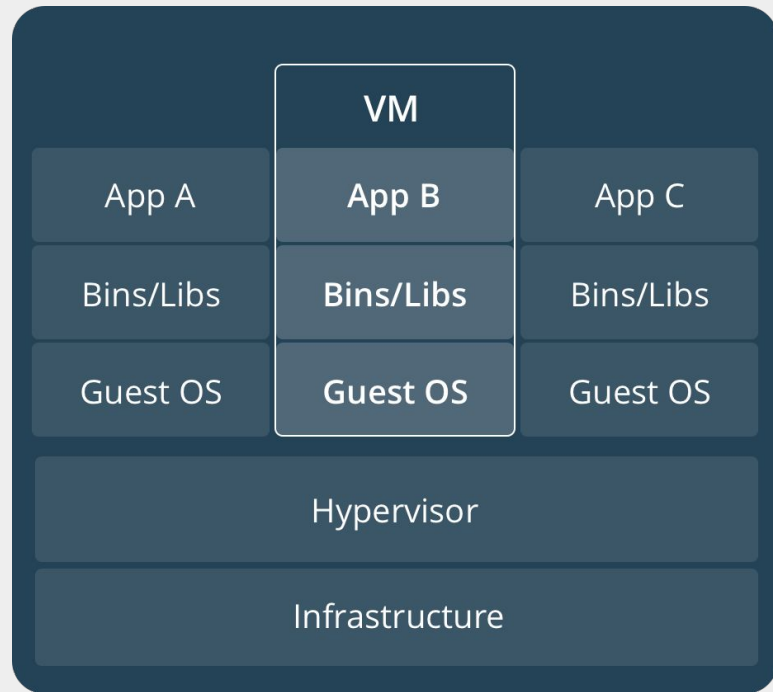
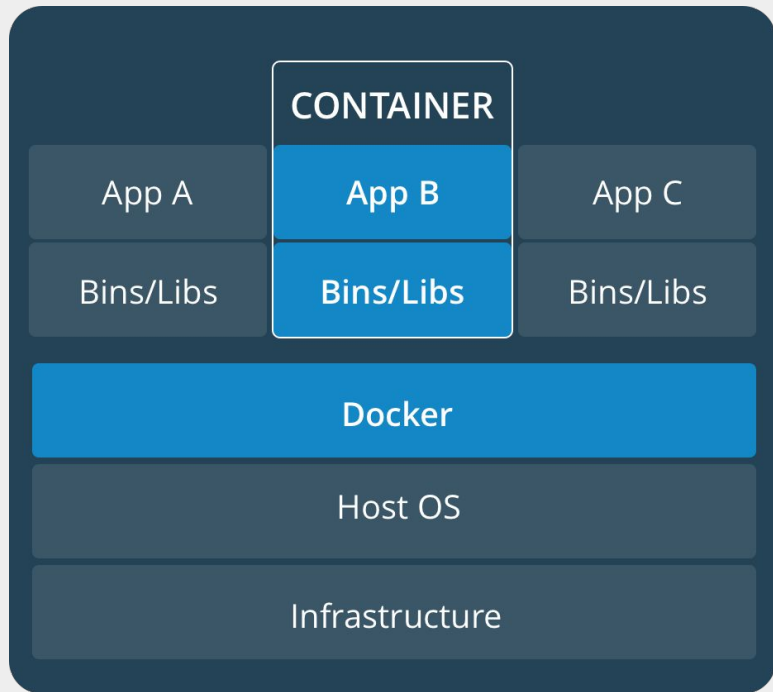


O que são contêineres?

- Os contêineres são mecanismos fáceis para construir, distribuir e executar componentes de software compartimentados.
- Geralmente, os contêineres empacotam todos os binários e bibliotecas essenciais para a execução de um aplicativo.
- Os contêineres reservam seu próprio sistema de arquivos, endereço IP, interfaces de rede, processos internos, namespaces, bibliotecas do sistema operacional, binários do aplicativo, dependências e outras configurações do aplicativo.



A diferença entre máquinas virtuais e contêineres



Contêineres e máquinas virtuais juntos

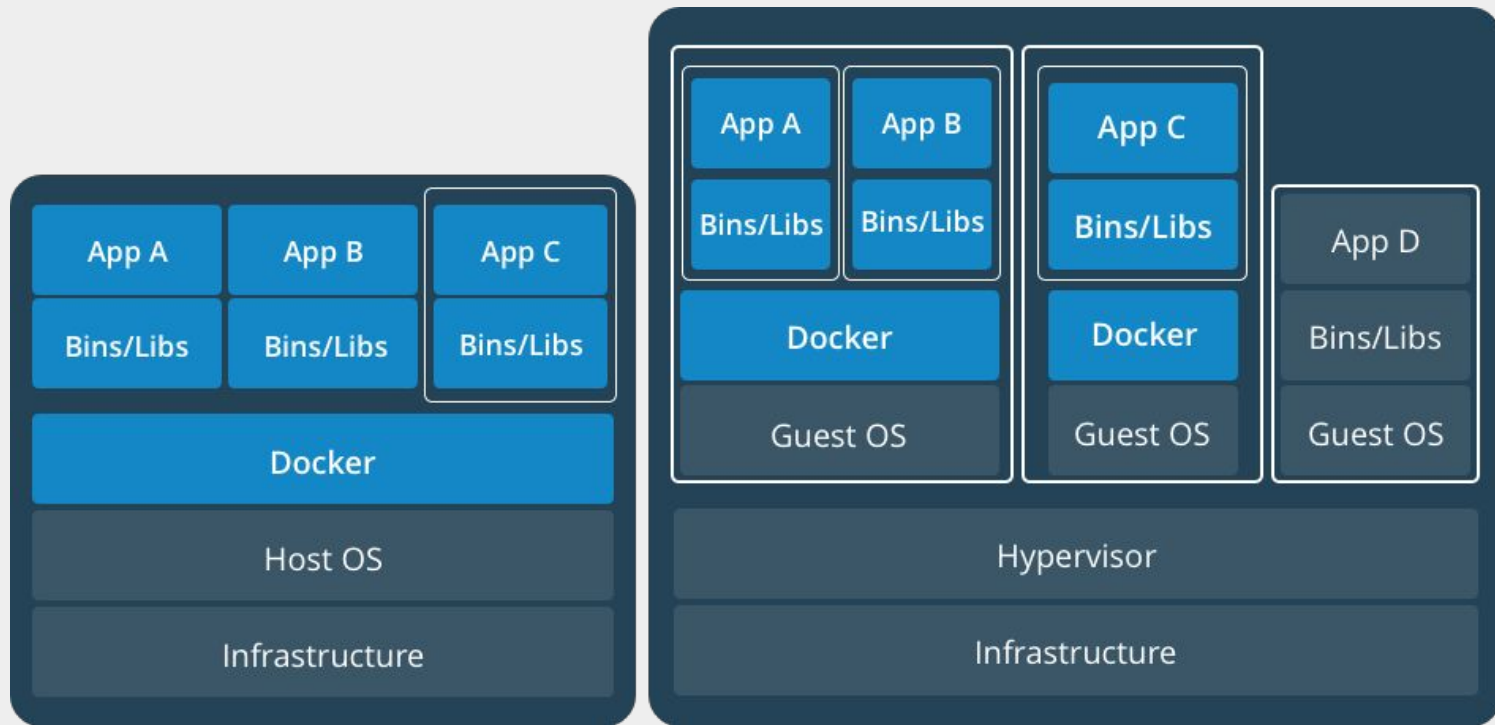


Imagem vs. Container

Essa é a distinção mais importante para entender Docker. Confundir os dois é a principal fonte de erros para quem está começando.



Imagem

Um **template somente leitura** que descreve o sistema de arquivos e as configurações do ambiente. É como uma receita ou uma planta baixa. Não executa nada por si só.

Criada a partir de um `Dockerfile` com o comando `docker build`.



Container

Uma **instância em execução** de uma imagem. É o processo real, com seu próprio sistema de arquivos isolado, rede e ciclo de vida. Pode ser iniciado, pausado e removido.

Criado a partir de uma imagem com o comando `docker run`.



A Analogia Perfeita

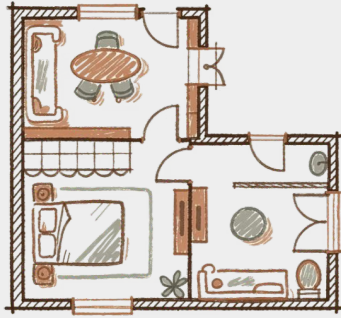


Imagem = Planta Baixa

A planta define como a casa deve ser construída. Você pode construir várias casas idênticas a partir da mesma planta. A planta em si não é uma casa.



Container = Casa Construída

A casa é a planta em execução. Você pode ter múltiplas casas (containers) a partir de uma única planta (imagem), cada uma com seus próprios moradores e histórico.



A distinção mais importante

Compreender a diferença entre imagem e container é o primeiro passo para trabalhar com Docker com segurança e eficiência.

	Imagem	Container
Analogia	Receita de bolo	O bolo assado
Estado	Imutável	Tem processo, memória, estado
Origem	Construída via Dockerfile	Instanciada a partir de uma imagem
Ciclo de vida	Persiste até ser deletada	Para quando o processo termina

📄 Você pode ter uma imagem e zero containers, ou uma imagem e cem containers rodando simultaneamente, todos idênticos.



O Dockerfile: A Receita do Ambiente

O `Dockerfile` é um arquivo de texto que descreve, passo a passo, como construir uma imagem Docker. Cada instrução gera uma **camada** no sistema de arquivos da imagem. Essas camadas são cacheadas pelo Docker para acelerar rebuilds.



Ao versionarmos o `Dockerfile` junto com o código-fonte, garantimos que qualquer desenvolvedor ou servidor de CI possa reproduzir exatamente o mesmo ambiente, eliminando a dependência de configurações manuais.



Instruções Essenciais do Dockerfile

FROM

Define a imagem base. Todo Dockerfile começa com FROM. Ex.: `FROM node:18-alpine`

WORKDIR

Define o diretório de trabalho dentro do container. Todos os comandos seguintes rodam a partir daqui.

COPY

Copia arquivos do host para dentro da imagem. `COPY . .` copia tudo do contexto atual.

RUN

Executa um comando durante o build. Usado para instalar dependências: `RUN npm install`

EXPOSE

Documenta qual porta o container escuta. É declarativo — não publica a porta automaticamente.

CMD

Define o comando padrão ao iniciar o container. Ex.: `CMD ["node", "server.js"]`



Dockerfile — a receita escrita em texto

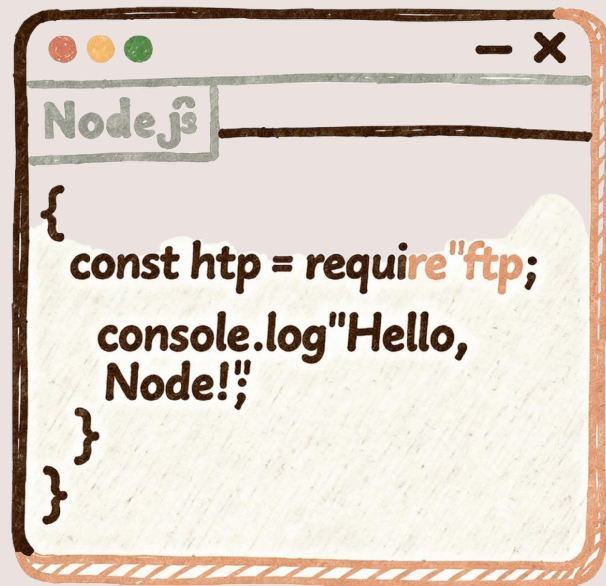
O Dockerfile é um arquivo de texto com instruções que o Docker executa sequencialmente para construir uma imagem. Cada instrução gera uma camada no sistema de arquivos da imagem.

Instrução	O que faz	Exemplo
FROM	Imagem base de partida	FROM node:20-alpine
WORKDIR	Diretório de trabalho no container	WORKDIR /app
COPY	Traz arquivos do host para o container	COPY . .
RUN	Executa comando durante o build	RUN npm ci
EXPOSE	Documenta a porta usada (só documentação!)	EXPOSE 3000
CMD	Comando padrão ao iniciar o container	CMD ["node", "src/index.js"]



Dockerfiles na Prática

Vamos analisar Dockerfiles reais do projeto **PetFood**, um backend em Node.js/Express e um frontend em React. Prestaremos atenção especial a estratégias de otimização que fazem diferença no dia a dia



Backend PetFood — Node.js/Express containerizado

```
# Imagem base leve
FROM node:20-alpine

# Diretório de trabalho
WORKDIR /app

# Copia só o manifesto primeiro
# – aproveita cache de camadas
COPY package*.json ./

RUN npm ci --omit=dev

# Código-fonte separado para não
# invalidar cache de dependências
COPY . .

EXPOSE 3000

CMD ["node", "src/index.js"]
```

💡 Por que copiar package*.json antes?

Se o código mudar mas as dependências não, o Docker reutiliza a camada de npm ci do cache, economizando minutos de build a cada iteração do desenvolvimento.

✅ Essa ordem é uma das otimizações mais importantes que você pode aplicar em qualquer Dockerfile Node.js.



Estratégia de Cache de Camadas

A ordem das instruções no Dockerfile não é aleatória, ela é uma decisão de performance crítica. O Docker reutiliza camadas cacheadas sempre que uma instrução não mudou.

1

COPY package*.json

Copiamos só os manifests de dependência primeiro. Essa camada só invalida o cache quando as dependências mudam.

2

RUN npm ci

Instala as dependências. Como a camada anterior raramente muda, esse passo caro é quase sempre servido do cache.

3

COPY ..

Copia o código-fonte. Muda a cada commit — mas as dependências já estão cacheadas acima.

- ✅ Com essa estratégia, rebuilds após mudanças de código ficam até 10x mais rápidos em projetos reais, pois o `npm install` não é reexecutado.



Por Que a Ordem Importa?

✗ Ordem Ingênua

```
COPY . .  
RUN npm install
```

Qualquer mudança em qualquer arquivo de código (mesmo um comentário) invalida o cache do `npm install`. Resultado: download completo das dependências a cada build.

✓ Ordem Otimizada

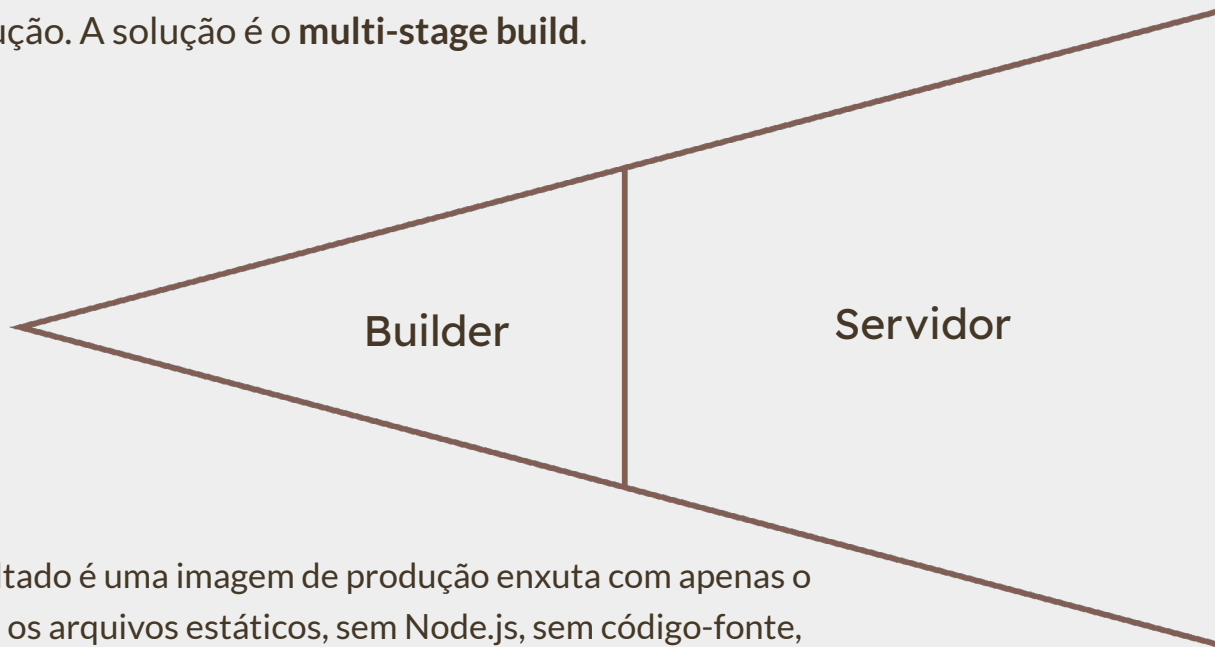
```
COPY package*.json ./  
RUN npm install  
COPY . .
```

O cache do `npm install` só é invalidado quando `package.json` muda. Mudanças no código-fonte não reexecutam a instalação de dependências.



Multi-Stage Build para o Frontend React

O React precisa de um passo de compilação (`npm run build`) que gera arquivos estáticos. Mas você não quer carregar o Node.js e todas as `node_modules` em produção. A solução é o **multi-stage build**.



O resultado é uma imagem de produção enxuta com apenas o nginx e os arquivos estáticos, sem Node.js, sem código-fonte, sem `node_modules`. A imagem final pode ser até **20x menor** do que uma imagem ingênua.



Dockerfile do Frontend React

```
# ===== ESTÁGIO 1: Build =====  
FROM node:18-alpine AS builder  
  
WORKDIR /app  
  
COPY package.json package-lock.json ./  
RUN npm ci  
  
COPY . .  
RUN npm run build  
# Gera a pasta /app/dist com os arquivos estáticos  
  
# ===== ESTÁGIO 2: Servidor de Produção =====  
FROM nginx:alpine  
  
# Copia APENAS o resultado do build  
COPY --from=builder /app/dist /usr/share/nginx/html  
  
EXPOSE 80  
  
CMD ["nginx", "-g", "daemon off;"]
```



Frontend PetFood — React precisa de uma estratégia diferente

⚠ React precisa de Node para compilar, mas em produção só precisamos dos arquivos estáticos gerados pelo build.

```
# Stage 1: compilar o React com Node
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build    # gera /app/dist

# Stage 2: servir os arquivos estáticos com nginx
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

Stage 1 — descartado

Node + npm + código-fonte: usado apenas para compilar, não vai para a imagem final.

Imagem final — nginx

Apenas nginx + arquivos estáticos: **menos de 30 MB**, segura e pronta para produção.



Benefícios do Multi-Stage Build

Imagem Menor

A imagem final não inclui Node.js, npm ou `node_modules`. Apenas o nginx e os arquivos HTML, CSS e JS compilados.

Superfície de Ataque Reduzida

Menos ferramentas na imagem significa menos vetores de vulnerabilidade. O container de produção não tem compiladores nem gerenciadores de pacotes.

Deploy Mais Rápido

Imagens menores são transferidas mais rápido entre registries e servidores, acelerando pipelines de CI/CD e deploys.



O Arquivo .dockerignore

O Que É?

Funciona como o `.gitignore`: lista arquivos e pastas que devem ser **excluídos do contexto de build**. O Docker envia todo o contexto para o daemon antes de processar o Dockerfile.

Exemplo Típico

```
node_modules
.git
.env
*.log
dist
coverage
.DS_Store
```

Por Que É Importante?

Performance: Sem `.dockerignore`, a pasta `node_modules` inteira é enviada ao daemon antes do build, podendo ter centenas de MB desnecessários.

Segurança: Evita que arquivos `.env` com credenciais sejam acidentalmente copiados para dentro da imagem.

Correção: Garante que o `node_modules` local não sobrescreva as dependências instaladas dentro do container.



.dockerignore — tão importante quanto .gitignore

```
node_modules # nunca copie - será instalado
              # dentro do container
.env          # segredos não pertencem à imagem
.git          # histórico do repo é desnecessário
*.log        # logs locais não vão a lugar nenhum
dist         # será gerado pelo build
```

Por que isso importa?

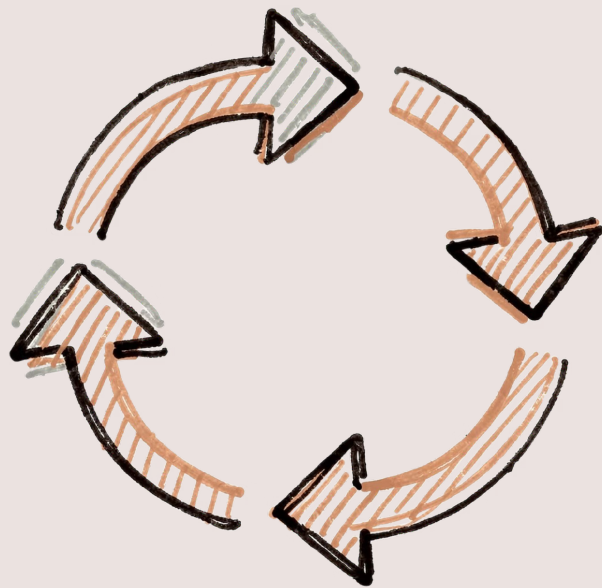
Sem `.dockerignore`, o comando `COPY . .` levaria o diretório `node_modules` inteiro para dentro do container, incluindo binários compilados para o sistema operacional errado, aumentando o tamanho da imagem e causando erros silenciosos.

- ⊗ Nunca inclua arquivos `.env` em imagens Docker. Segredos devem ser injetados em runtime via variáveis de ambiente ou sistemas como Docker Secrets.



Ciclo de Vida Básico de um Container

Com o Dockerfile pronto, é hora de aprender os comandos essenciais para construir, executar e inspecionar containers no dia a dia. Esses são os comandos que você usará em todo projeto Docker.



docker build: Construindo a Imagem

```
# Construir uma imagem com tag
docker build -t petfood-backend:1.0 .

# O ponto (.) indica o contexto de build (diretório atual)
# -t define o nome e a tag da imagem
```

O que acontece por baixo

O Docker lê o Dockerfile, processa cada instrução sequencialmente, verifica o cache para cada camada e gera uma imagem identificada por um hash SHA256 único.

Tags e versionamento

Use tags semânticas como `:1.0`, `:latest` ou `:production`. A tag `latest` é o padrão quando nenhuma é especificada, mas evite depender dela em produção.



docker run: Iniciando um Container

Modo básico

```
docker run petfood-backend:1.0
```

Com publicação de porta: host:container

```
docker run -p 3000:3000 petfood-backend:1.0
```

Em segundo plano (detached) com nome personalizado

```
docker run -d --name backend -p 3000:3000 petfood-backend:1.0
```

Passando variáveis de ambiente

```
docker run -d -p 3000:3000 -e NODE_ENV=production petfood-backend:1.0
```

❗ A flag `-p 3000:3000` mapeia a porta 3000 do host para a porta 3000 do container. Sem esse mapeamento, a aplicação roda isolada e inacessível externamente.



Inspecionando e Gerenciando Containers

docker ps

Lista containers em execução. Use `docker ps -a` para ver todos, incluindo os parados. Mostra ID, imagem, status, portas e nome.

docker logs

Exibe os logs de saída (stdout/stderr) de um container. Use `docker logs -f backend` para acompanhar em tempo real (follow mode).

docker stop

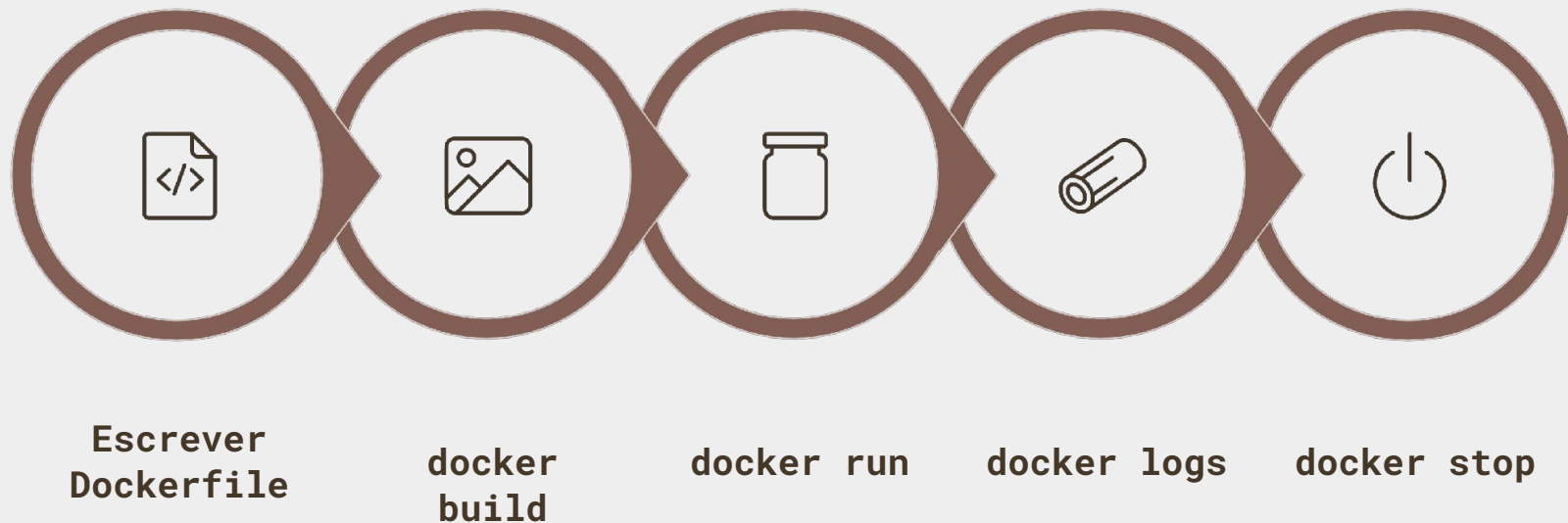
Para um container graciosamente, enviando sinal SIGTERM e aguardando 10 segundos antes de forçar com SIGKILL. Use `docker stop backend`.

docker rm

Remove um container parado. Use `docker rm backend`. Para remover automaticamente ao parar, adicione `--rm` ao `docker run`.



Fluxo Completo: Do Dockerfile ao Container Rodando



Esse fluxo se repete em todo projeto Docker. Com o tempo, ele se torna automático, e ferramentas como Docker Compose e pipelines de CI/CD automatizam ainda mais esses passos, mas o fundamento permanece o mesmo.



Construir, rodar, inspecionar, parar

```
# Construir a imagem
docker build -t petfood-backend .

# Rodar o container
# (porta 3000 do host → porta 3000 do container)
docker run -d -p 3000:3000 petfood-backend

# Ver containers em execução
docker ps

# Ver logs do container
docker logs <container_id>

# Parar o container
docker stop <container_id>
```

-d

Detached: roda em background, libera o terminal.

-p host:container

Mapeia porta do host para porta interna do container.

-t (no build)

Tag: nomeia a imagem gerada para fácil referência.



Comparativo: Antes e Depois do Docker

Situação	Sem Docker	Com Docker
Onboarding de novo dev	Horas instalando dependências e resolvendo conflitos	<code>docker run</code> e pronto em minutos
Paridade de ambiente	Diferenças entre dev, staging e produção	Mesma imagem em todos os ambientes
Versão do Node	Depende do que está instalado no SO	Definida no FROM do Dockerfile
Deploy	Scripts manuais, documentação desatualizada	Imagem versionada e reproduzível
Tamanho da imagem de produção (React)	Centenas de MB com Node.js	~25 MB com nginx + multi-stage



node:20-alpine vs. node:20 — qual a diferença?

	node:20	node:20-alpine
Tamanho	~900 MB	~130 MB
Base	Debian	Alpine Linux
Uso recomendado	Desenvolvimento local	Produção

- ✔ Menos tamanho = menos superfície de ataque, menos custo de transferência e deploy mais rápido. Alpine é o padrão de produção adotado pelo mercado.



Boas Práticas Resumidas



Use imagens alpine

Prefira `node:20-alpine` a `node:20`. São muito menores e têm menos vulnerabilidades.



Use npm ci

Em vez de `npm install`, use `npm ci` para builds reproduzíveis baseadas no `package-lock.json`.



Sempre use .dockerignore

Exclua `node_modules`, `.env` e `.git` para builds mais rápidos e seguros.



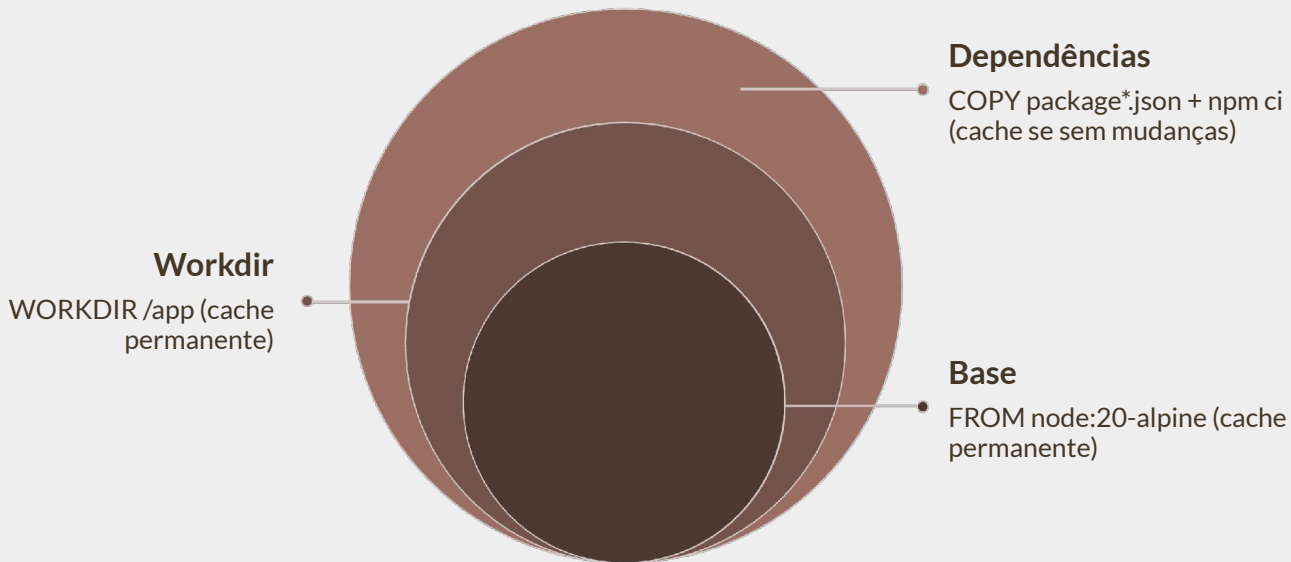
Otimize a ordem das camadas

Coloque o que muda menos primeiro para aproveitar o cache e acelerar rebuilds.



Como o Docker reutiliza trabalho

O Docker constrói imagens em camadas. Cada instrução no Dockerfile gera uma camada, e o Docker cacheia cada uma delas. Se uma camada não mudou desde o último build, ela é reutilizada, tornando builds subsequentes muito mais rápidos.



- ❑ **Regra prática:** Coloque o que muda com **menos frequência** no topo do Dockerfile. Coloque o que muda com **mais frequência** embaixo. Isso maximiza o reaproveitamento do cache e acelera seus builds.



**O que você mudaria se pudesse
começar o semestre de novo com essa
ferramenta?**

"O Docker resolve o problema de ambiente. Mas esse problema existia no seu projeto esse semestre. O que sua equipe fez para lidar com ele sem Docker?"



Hoje: um container. Amanhã: um sistema inteiro.

O que vimos hoje

- O problema que Docker resolve e por que importa profissionalmente
- Imagem vs. container — a distinção fundamental
- Dockerfile do PetFood — backend Node.js e frontend React
- Ciclo de vida básico de um container

Amanhã — terça-feira

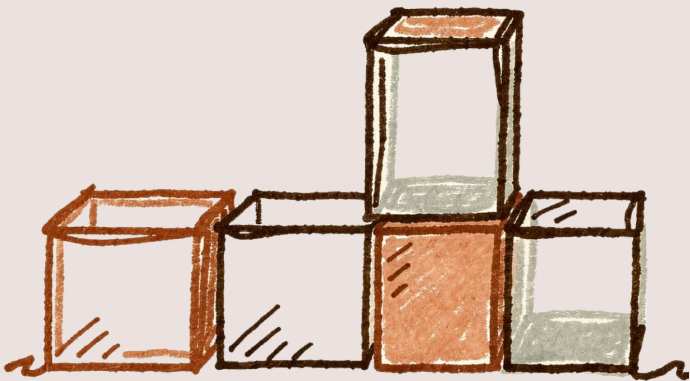
O PetFood tem três serviços: **frontend React**, **backend Node.js/Express** e **banco PostgreSQL**. Como subir os três juntos com um único comando?

Docker Compose

Essa é a resposta, e é o que veremos na próxima aula.



Referências e próximos passos



Documentação Oficial

docs.docker.com — referência completa de comandos, Dockerfile e boas práticas. Sempre atualizada e com guias de início rápido.



Pratique agora

Clone o repositório do PetFood, escreva o Dockerfile do backend seguindo o modelo da aula e rode `docker build + docker run` localmente.



Docker Hub

hub.docker.com — repositório público de imagens. Explore as imagens oficiais do Node, nginx e PostgreSQL que usamos no PetFood.



Próximos Passos

Com esses fundamentos, você já consegue containerizar aplicações Node.js e React. O ecossistema Docker vai muito além, e os próximos passos naturais são:

Docker Compose

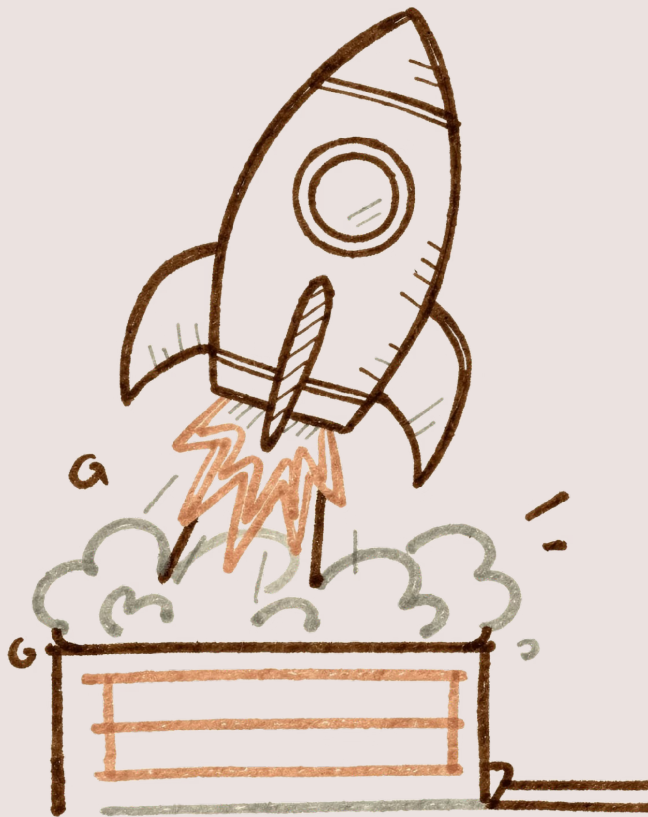
Orquestre múltiplos containers (backend + frontend + banco de dados) com um único arquivo `docker-compose.yml`.

Volumes e Redes

Persista dados além do ciclo de vida do container e configure comunicação entre serviços com redes Docker.

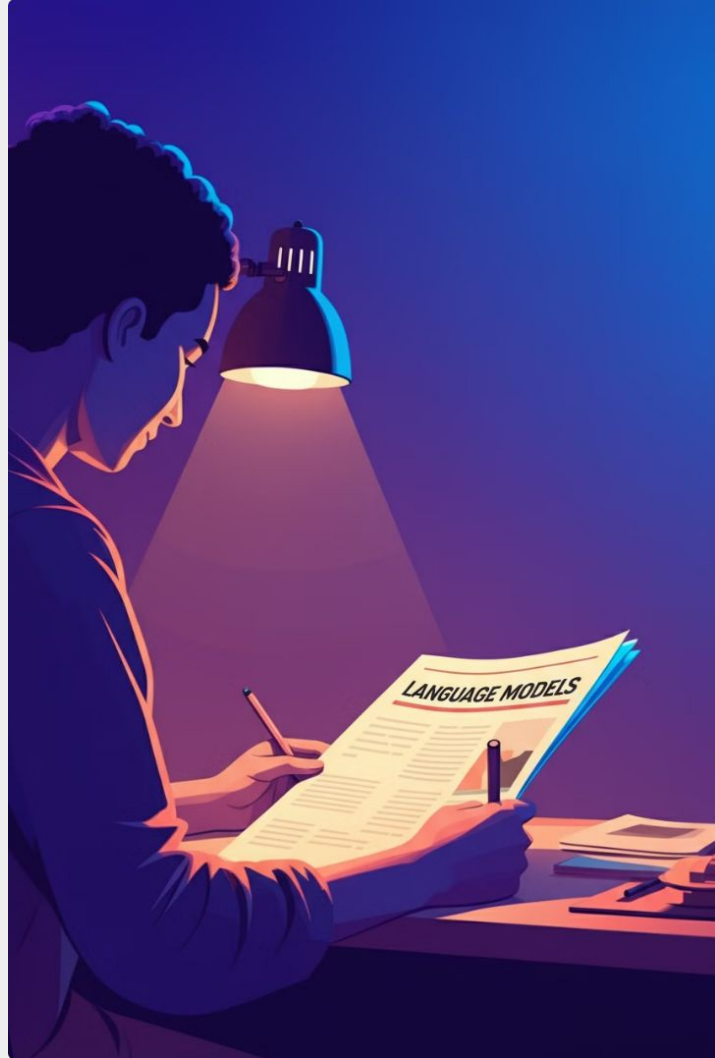
Registry e CI/CD

Publique imagens no Docker Hub ou GitHub Container Registry e automatize builds com GitHub Actions.



Próximos passos

- Eng. Software em Dimensões
 - Cap. 2, seção 2.5.1 — Fator ônibus e socialização do conhecimento via Code Review.
 - Cap. 3 (seções 3.1–3.7): Versionamento, Git, PRs, Code Review, licenças
 - Cap. 14 (seções 14.1–14.8): Arquitetura, C4 Model, qualidade arquitetural
 - Cap. 19 (seções 19.1–19.4): Manutenção, refactoring, dívida técnica, documentação
- Eng. Software Moderna
 - Cap. 9 — Refactoring, Catálogo, code smells, prática
 - Cap. 10 — DevOps, Controle de versões, integração contínua



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Do Código ao Container